

Voice-to-Form Health Clinic

2-Page Recording Presentation Brief | Azure Cloud Project

Suggested length: about 5-7 minutes. Speak naturally; you do not need to read every word exactly.

1. Opening - What the project does

Hello, my project is called **Voice-to-Form Health Clinic**. The goal is to make patient intake easier by allowing a patient to speak instead of typing a long form. The user opens the website, signs in, chooses a language, records a health statement, and submits the recording. Azure then processes the audio in the background, converts speech to text, translates it to English when needed, creates the final patient-intake PDF, and makes that PDF available to the user.

Show: the public site at <https://voicetoform.amtcloud.site>, then the welcome page and language selection.

2. High-level Azure architecture

The solution is built completely in Microsoft Azure. At the front, I use my custom domain with **Azure Traffic Manager**. Traffic Manager uses priority routing: **Canada Central** is the primary region and **Canada East** is the failover region. In each region, traffic first reaches **Application Gateway with Web Application Firewall**, then goes to **API Management**, and then to the regional **Azure App Service** that hosts my Node.js web application.

This design gives me a secure public entry point, an API control layer, and regional resiliency. My main public address stays the same even if Traffic Manager switches the user from Central to East.

Show: your architecture diagram, then the Central and East resource groups, and finally Traffic Manager with Central Priority 1 and East Priority 2.

3. Frontend and application code

I developed the frontend in **Visual Studio Code**. The web project contains **index.html**, **styles.css**, and **app.js** inside the public folder, plus **server.js** and **package.json**. The Node.js backend provides routes such as **POST /api/submit**, **GET /api/status/:submissionId**, **GET /api/download/:submissionId**, and **GET /api/health**. I tested the application locally on port 8080 before deploying it to Azure App Service.

*Show: VS Code Explorer with the web project structure, then briefly open **server.js**.*

4. Voice processing and asynchronous backend

When a patient submits audio, the Web App does not keep the browser waiting for all AI work to finish. It saves the recording in the **temp-intake** Blob container, creates a submission record in the **submissions** table, and sends a message to the **voicetoform-queue**. An **Azure Function** named **QueueWorker** is triggered by that queue message. The function performs the background processing, uses Azure AI Speech for speech-to-text, uses Azure AI Translator when translation is required, prepares the final document, stores the PDF in the **pdfs** container, and updates the submission status to completed.

The frontend polls the status endpoint. When processing finishes, the page changes from processing to **Your form is ready** and displays the **View completed PDF** button.

Show: Storage Browser, the queue, QueueWorker Monitor, then demonstrate one submission from recording to completed PDF.

Voice-to-Form Health Clinic - Recording Brief

Security, resiliency, monitoring, challenges, and conclusion

5. Security and authentication

Security is an important part of the project. The public site uses **HTTPS**, and Application Gateway provides **WAF** protection. I use **Microsoft Entra External ID** with App Service Authentication, also called Easy Auth, for patient sign-in. The project also uses **Azure Key Vault** for key and certificate management. Storage encryption uses a customer-managed key, and managed identities with Azure RBAC are used so Azure services can access Storage without putting storage keys directly in the application code.

Show: Authentication configuration, Key Vault certificate, and the secure custom-domain sign-in flow. Do not show secrets, access tokens, or storage keys.

6. Canada East disaster recovery and failover

To demonstrate high availability, I created a second regional stack in **Canada East**, including another Application Gateway, API Management instance, App Service, Function App, and monitoring resources. Traffic Manager normally returns the Canada Central gateway. During my failover test, I disabled the Central Traffic Manager endpoint and DNS changed to the East Application Gateway at **40.86.207.48**. The application still loaded through the same custom domain. After I re-enabled Central, DNS returned to the Central gateway at **4.174.241.133**. This proved both failover and failback.

Show: Traffic Manager endpoints, Central DNS proof, East failover DNS proof, and the site working on the custom domain during failover.

7. Monitoring and observability

For monitoring, I use **Application Insights**, **Log Analytics**, Azure Monitor, Function monitoring, and Application Gateway backend health. These tools help me see requests, failures, performance, function executions, and whether regional backends are healthy.

Show: Application Insights or Live Metrics, QueueWorker successful invocation, and Application Gateway backend health showing Healthy.

8. Important problems I solved

Several issues helped me understand the architecture better. The East Web App originally received a **403 AuthorizationFailure** when accessing Storage, which I fixed by assigning Blob, Queue, and Table Data Contributor roles to its managed identity. I also fixed a **QueueNotFound** problem caused by using the wrong queue name. Application Gateway initially marked the East APIM backend unhealthy because authentication returned HTTP 401, so I created a custom health probe that accepts that authenticated response. Finally, Easy Auth originally redirected users to the East azurewebsites.net hostname during failover. I configured the forward proxy setting to use **X-Original-Host**, which allowed the browser to remain on **voicetoform.amtcloud.site** after sign-in.

9. Final demo and closing

For the final demo, I start at the custom domain, sign in, choose a language, record a short patient statement, submit it, show the processing status, and then open the completed PDF. This demonstrates the complete path from the browser through Azure networking and security, into Storage and Queue processing, through Azure Functions and AI services, and back to the patient as a finished document.

In summary, this project demonstrates **cloud application development, security, serverless processing, Azure AI integration, monitoring, and multi-region disaster recovery**. The final solution is functional, scalable, secure, and resilient, and it gives me practical experience connecting many Azure services into one end-to-end architecture.

Finish by showing the completed PDF and your architecture diagram.

Recording tip: keep your cursor moving with the explanation. Show one screen per section instead of staying on the same Azure page.